Document	**P3109R0**
Date	**2024/02/12**
Reply To	**Lewis Baker <lewissbaker@gmail.com>** **Eric Niebler <eric.niebler@gmail.com>** **Kirk Shoop <kirk.shoop@gmail.com>** **Lucian Radu Teodorescu <lucteo@lucteo.ro>**
Audience	**LEWG**

P3109R0: A plan for std::execution for C++26

Abstract

P2300 introduces the sender/receiver framework, laying the foundational model for asynchrony in C++ and the standard library. However, before we ship P2300 and freeze its design in the standard, we should ensure that all potentially-breaking design questions are fully resolved, and that the initial release includes a sufficiently well-rounded set of utilities to be generally useful.

In this paper we aim to identify and prioritize these critical work items that need to be worked on in the C++26 cycle, on top of what's already in P2300. Additional features that we identify as secondary priorities, and don't need to ship with the

initial release of sender/receiver are described in P3118. However, we do not exclude the possibility of working on those features targeting the initial release.

A summary of existing facilities defined by P2300 is listed in Appendix A for reference. They are also listed in an updating experimental section in Cppreference.

Executive Summary

The following work items should be prioritized to address potentially-breaking design questions in P2300:

- **Removing ensure_started() algorithm, replacing it with a new async scope facility.**
  - `ensure_started()` has some foot-guns that make it difficult to use correctly/safely
  - we should replace `ensure_started()` with a new async-scope abstraction that provides equivalent functionality but in a way that has fewer footguns
  - Needs a paper.
- **Remove dependency on tag_invoke**
  - Current wording of P2300R7 uses tag_invoke()-based customisation points to define the core concepts as well as customisable algorithms.
  - tag_invoke relies on dispatching all customisation points through a single ADL function, relying on tag-dispatch to select the right overload, which has raised compile-time scalability concerns.
  - P2999R3 proposed replacing algorithm customisation with a new domain-based customization design that relies on member-functions on a type obtained from an environment - already forwarded by LEWG.
  - P2855 proposes changing the core concepts to instead be member-function based. This change needs to be made before we ship as it is a breaking change. Needs a new paper revision.
- **Changes to the 'queryable' concept used for receiver environments and sender attributes**
  - Extend the queryable concept with the ability to enumerate the queries it provides values for. This is needed to address limitations in `split()` and `ensure_started()` algorithms that currently prevent them from forwarding sender attributes of the wrapped sender.
  - Explore whether the 'queryable' facility should be made more generally available for other use-cases that want to be able to pass a bag of properties. If so, we may also want to consider shipping the concept with additional facilities for creating/manipulating queryable objects.
  - Needs papers.
- **Explore change to get_stop_token() requirements that would allow reduced synchronization overhead for stop callbacks**
  - It may be possible to reduce the amount of synchronization needed to support cancellation by restricting sender operations to registering a single stop-callback at a time with a stop token obtained from `std::execution::get_stop_token()`.
  - This would potentially make cancellable algorithms more efficient.
  - Making this change later would be a breaking change for user-authored senders that register multiple stop-callbacks, either

directly or indirectly by passing the stop-token to multiple child operations, each of which might register their own stop callbacks.
    - Needs a paper.
- **Improvements to the design of std::execution::run_loop class**
    - The run_loop class provides a simple scheduler implementation that allows you to manually drive an event loop by calling a `run()` method that executes work scheduled to it until `finish()` is called. It is used by `sync_wait()`.
    - The current design has a very limited use - you can only call `run()` once. If there are items queued to it after you call `finish()` then there is no way to execute them. This makes it difficult or impossible to use in scenarios that are not roughly equivalent to `sync_wait()`.
    - We should consider changing the design to make it more generally useful. e.g. by allowing repeated calls to drive the event loop.
    - Needs a paper.
- **Heap-allocated senders need support for custom allocators**
    - The algorithms `ensure_started()`, `split()` and `start_detached()` all need to dynamically allocate memory for the operation-state internal to the operation.
    - These algorithms do not currently provide any way to customize the allocation.
    - We should update these algorithms to allow passing a custom allocator that would be used for the allocation.
    - Depending on the chosen design this could either be a breaking change or added later.
    - Needs a paper.

The following work items should be prioritized to ensure that the initial ship vehicle for `std::execution` includes enough facilities to be generally useful out of the box:

- **Async Scope**
    - Enables launching a dynamic number of async operations that all use a given resource and then ensuring that all of those operations complete before the resource is destroyed.
    - To support this use-case, we propose a facility that represents a scope that exits after all nested async-functions have completed.
    - This facility should allow us to remove the unsafe `ensure_started()` algorithm (see Remove `ensure_started()` algorithm section).
    - Paper forthcoming.
- **Coroutine task type**
    - We expect the most common use of senders will actually be from within coroutines.
    - It would be a real disservice for users to have to wait until C++29 before they have an async coroutine task type out of the box that worked with the new async model proposed in P2300.
    - Needs a new paper - the design described in P1056 predated sender/receiver and does not support many of the features required for interoperability with senders.
- **System Execution Context**
    - Several of the facilities in P2300 require a scheduler but the only scheduler provided by P2300 is the one from `std::execution::run_loop`.
    - We should provide a scheduler out of the box that allows users to make use of the underlying platform's system thread-pool for

executing sender-based code in a portable way across multiple threads.
    - Without this, users will need to go to third party libraries for thread-pools, or write their own, in order to write async code that utilizes multiple CPU cores.

This paper, along with P3118, will introduce our suggested modifications and extensions.
This paper includes topics which are high priority for **C++26**, and P3118 will present lower priority fixes and additions which can be done post initial delivery.

The modifications and additions are split into priorities. This paper includes all the P0 topics.
Classification is as follows:

- P0 - highest priority - things we can't change after we ship.
- P1 - high priority - we should strive to ship this in the first version of std::execution to improve the user-experience, but it can be added later if needed
- P2 - nice to have - we should continue to make progress on this in parallel with other work, but it is fine to target a subsequent ship-vehicle.

To summarize, the modifications and additions suggested in this paper:

Paper	Description	Change type	Details
	Async Scope (+ removing ensure_started)	Design addition	High priority design addition, which will allow management of multiple fire-and-forget executions to be done safely.
P2855	Member-function-based customization	Design modification	Remove the tag_invoke ADL-based CPs
P3121	Fine tune the "queryable" concept, (potentially) add generalized Environment/Queryable Facilities	Design modification (generalization)	We may be able to define the CPs only, and deliver full utility later
-	Reducing the synchronization overhead of cancellation	Optimization	By exploring changes in get_stop_token() requirements
-	Design changes to the run_loop	Design modification	Explore modifications to the API of "run_loop"
-	Heap-allocating sender algorithms need allocator support	Design addition	Add support for allocators. Having this post initial revision will be a potential ABI break.
-	Coroutine task type	Design addition	Support what is expected to be a main use case
P2079	System Execution Context	Design addition	Adding a system-wide scheduler that allows to spawn concurrent

			work. Required for a basic usage of S/R for typical concurrency problems.
-	Clarify destruction order of algorithms	Design modification	Add lifetime requirements on objects or sub-objects passed to algorithms

Library Modifications and Additions

Library Design Modifications

Remove `ensure_started()` algorithm (P2519)

Senders are generally assumed to be safe to destroy at any point. It is common to have algorithms that senders can be composed with that are not guaranteed to connect/start their child senders.

However, `ensure_started()` returns a sender that owns work that is potentially already executing asynchronously in the background.

If this `ensure_started()` sender is destroyed without being connected/started then we need to specify the semantics of what will happen to that already-started asynchronous work. There are four common strategies for what to do in this case:

1. Block in the destructor until the asynchronous operation completes - can easily lead to deadlock.
2. Detaching from the async operation, letting it run to completion in the background - makes it hard to implement clean shut-down.
3. Treat it as undefined-behavior - this behavior is user-hostile.
4. Terminate the program - the strategy that std::thread takes - also user hostile.

The current `ensure_started()` wording chooses option 2 as the least worst option, but all of the options are generally bad options.

We should remove `ensure_started()` from P2300 before it is merged into the IS.

To provide equivalent functionality of being able to eagerly spawn work, we should provide the async_scope abstraction, that allows spawning eager work, while still allowing the work to be joined, preserving structured-concurrency, even if the returned sender is never connected/started.

See the section on Async Scope for work relating to the replacement facility.

Move to member-function-based customization (P2855)

The current specification of P2300 defines the customization-points for the core concepts in terms of `tag_invoke`, a generic ADL-based customization point mechanism.

The use of a single ADL name for dispatching to a customization of any algorithm is powerful, but also problematic for compile-times as it makes the overload-set sizes potentially much larger, since overloads for receiver methods (`set_value`, etc.) are considered in a call to `connect()`.

This paper proposes to instead move back to a more direct member-function-based API for defining implementations of the core concepts.

e.g. instead of a sender defining:

```
  template<typename Receiver>
 friend op_state<Receiver> tag_invoke(connect_t, my_sender&&
self,
                                      Receiver r) { /* … */
}
```

they can define the following member function:

```
  template<typename Receiver>
 op_state<Receiver> connect(Receiver r) && { /* … */ }
```

In combination with the changes introduced in P2999, which removes ADL-based customization of sender algorithms in favor of a member-function based customization on a customization domain object, there will be no more ADL-based customization points left in the design.

The result should be better compile times, and also a simpler interface for authors of senders, receivers and operation-states to implement.

This is a change to the definition of the core concepts and so is a change that we cannot make after we ship sender/receiver in a standard.

Generalised Environment/Queryable Facilities (P3121)

The `queryable` concept in P2300 defines the interface that “environments” provide. These environments are obtained either from a receiver or from a sender, and let you query properties of that environment to get additional information about the context of the caller (receiver) or of the operation (sender).

For example, environments are used to pass information about the current scheduler, allocator or stop-token to child operations via the receiver.

These environments generally are read-only and have reference semantics - they are more like handles to the environment - so that asking for the environment is cheap.

There are a number of improvements we should consider making to the queryable concept listed below. We need to make these changes before shipping `std::execution` as it will be difficult to change it later.

Enumerating queries

The queryable concept is currently missing the ability to enumerate which queries it provides values for. This makes it difficult to, for example, generically construct a new environment that holds a copy of each of the values of an arbitrary given environment.

Such a facility is needed by algorithms like `split()` and `ensure_started()` to be able to continue using the sender's attribute values after the original sender's lifetime has ended by taking a copy of the values of each attribute.

The shape of this might be as simple as requiring environments to have a nested type-alias that resolves to a type-list that contains the list of CPO types of queries it supports.

Such a type-alias would also allow making the queryable concept more selective, as it could check for the existence of this type-alias.

Remove dependency on tag_invoke

The current wording in P2300R7 defines a queryable object's interface in terms of calls to `tag_invoke()`.

The paper P2855 makes a case for removing the dependency of `std::execution` concepts on `tag_invoke()` and proposes replacing it with a `.tag_query()` member-function on the environment.

We could also consider other member-function based designs. e.g. using `operator[]` where the index is the query CPO.

Consider moving out of std::execution

The queryable concept is a general facility that may be useful outside of the use of `std::execution` and sender/receiver usage. It can potentially be used anywhere we want to be able to pass a bag of properties into some facility where the set of properties needs to be an open set.

If we expect it to be widely applicable in other domains, we should consider promoting the queryable concept, along with any related facilities, to the top level `std::` namespace and potentially consider putting it in a separate header.

Reducing the synchronization overhead of cancellation

The `stoppable_token` concept from P2300 allows the user to register a callback to be notified of a stop-request if some thread calls `request_stop()`on an associated stop-source. The current design of `std::stop_token`, as well as the proposed `std::in_place_stop_token` type introduced by P2300, allow multiple stop-callbacks to be registered to the same shared stop state.

An operation that obtains a stop-token from a receiver's environment using the `get_stop_token()` query is currently permitted to register multiple stop-callbacks to the same stop-token. This allows operations that are transparent to cancellation (i.e. that pass through stop-requests but do not generate their own) to just pass down the same stop-token obtained from the parent operation's receiver in the environment passed to all child operations.

However, not every sender algorithm needs to be able to register multiple stop-callbacks and in these cases we are potentially paying a performance penalty for a more complicated data-structure needed to maintain a list of multiple stop-callbacks in situations that don't need it.

One suggestion has been to instead restrict callers of the `get_stop_token()` environment query to only attaching a single stop-callback at a time to any given receiver's stop-token.

This can simplify the data-structure needed to implement the stop-token to one that is mostly lock free - only requiring blocking during deregistration of a callback that is concurrently executed on another thread.
Current implementations of multi-callback stop-tokens require acquiring a lock when adding, removing or invoking any stop-callbacks.

If we are to make such a change, we need to do it before we ship the `get_stop_token()` query as part of P2300 as trying to make this change later will be a breaking semantic change.

This change still requires some research to understand the impact of it on the complexity of algorithms with multiple child operations, but also to evaluate the performance benefit of this change on existing sender/receiver code-bases.

A sketch of an implementation of simpler stop-token that only allows a single stop-callback to be registered at a time is available here:
https://godbolt.org/z/sE43hPGW8

Note that we may still want to keep the `std::in_place_stop_token` with support for multiple stop-callbacks for cases where there are a potentially unbounded number of child operations. The main candidate for this would be in the implementation of a cancellable async-scope facility (see Async Scope).

Design changes to the `run_loop` class

The `std::execution::run_loop` class described in P2300 provides a simple scheduler implementation that schedules work in FIFO order on the thread that calls `run()`. This facility is used within sync_wait() to allow the calling thread to drive an event loop that can execute work while waiting for the operation to complete.

The design of the interface of `run_loop` is such that you can only drive the event-loop once by calling the `run()` member function. This `run()` function executes until some thread calls `finish()`, at which point the `run()` function will return and the `run_loop` object is now in the "finishing" state. The only thing you can do with such a `run_loop` object is destroy it.

This makes the `run_loop` class useful for `sync_wait()` but not much else. It is not possible to integrate this facility with other event-loops to allow them to incrementally run work scheduled using a `run_loop`.

We should revisit the design of `run_loop` to see if it can be modified to be usable in situations that require repeated calls to drive execution.

Alternatively, we could consider removing run_loop from the standard-library for this release, leaving it as an implementation detail of `sync_wait()` - the current wording hints at using `run_loop` but does not require it.

See the following issues for additional discussion:

- https://github.com/cplusplus/sender-receiver/issues/172
- https://github.com/cplusplus/sender-receiver/issues/120

Integration with other frameworks

One possible design for the run_loop to support integration with other event-loops would be to provide two methods:

- `ready_at()` - returns a cancellable sender that completes when an item is ready in the queue, it also completes to provide a time-point for when the next item is going to be ready. This allows scheduling the `run_loop` on another framework's scheduler without polling.
- `run_some()` - returns a cancellable sender that runs a finite number of queued items inline and then completes. This allows the `run_loop` queue items to be interleaved into another framework's scheduler

Clarify destruction order of algorithm resources

When a sender completes, the completion-signaling function is responsible for (eventually) destroying the operation-state object, which will eventually ensure that all of the resources of the operation are released.

However, it's not enforced that the operation-state of the just-completed operation is destroyed before the completion-signaling implementation goes on to execute logic that processes the result.

This can lead to an inconsistency in the order of destruction compared to normal functions, where resources held by a callee in its operation-state may still be alive while the caller executes some of its continuation rather than cleanup with nested lifetimes.

Whether an operation destroys the resources it holds before calling the completion-signaling function, or is deferred until the destructor of the operation state is called is currently not specified. We need to do an audit of the algorithms in P2300 to ensure that the destruction order is clear and makes sense for each of them.

Library Design Additions

Async Scope (P2519)

One pattern that we need to support is launching a dynamic number of async operations that all use a given resource and then ensuring that all of those

operations complete before the resource is destroyed.

`ensure_started()`, `start_detached()` is attractive, but problematic - can't join the work.

To support this use-case, we need an async-scope abstraction that represents a scope that waits until all nested async operations have completed. Destruction of any async resources used by those nested operations can then be scheduled for after the wait-operation has completed.

Async operations nested within an async-scope can either be launched eagerly or lazily:

- `spawn_future()` - eagerly starts the operation, returning a sender that can be used to observe the result
- `spawn()` - eagerly starts the operation, returning void - provides no way to observe the result
- `invoke()` - returns a sender that tracks completion of the operation via the scope, but starts the operation lazily when the returned sender is started

`spawn_future()` is similar to `ensure_started()`, but retains the ability to join the operation through the async-scope in the case that the sender is discarded without connecting and starting it.

`spawn()` is similar to `start_detached()`, but retains the ability to join the operation through the async-scope.

This facility should allow us to remove `ensure_started()` (see Remove `ensure_started()` algorithm section).

Design topics currently being worked on:

- Allocator support
  - The async-scope spawn() and spawn_future() operations will need to allocate storage for the operation-state. Users should be able to customize the allocation.
- Do we need cancellable/non-cancellable scopes?
  - Sometimes we may want an easy way to cancel all operations nested within an async-scope, or cancel only individual operations, or not cancel any of them.
- Publishing an async-scope paper independently of async resource lifetime management
  - We want to ship an async-scope facility with a design that is forwards compatible with async-sequences/async-cleanup so that we don't couple async_scope to accepting those other papers.
  - Goal: reduce procedural risk

There is existing implementation experience for async-scope facilities that solve some of these problems, but

Latest efforts on this can be found here:

- https://kirkshoop.github.io/async_scope/asyncscope.html
- https://github.com/ispeters/async_scope/pull/2/files

Example (using async resource lifetime management):

```
using namespace std::execution;

sender auto some_work(int work_index);
```

```
sender auto foo(scheduler auto sch) {
  return ex::use_resources( // with async resource
lifetime management
    [sch](counting_scope scope){
      return schedule(sch)
        | then([]{ std::cout << "Before tasks launch\n";
})
        | then(
          [sch, scope]{
            // Create parallel work
            for(int i = 0; i < 100; ++i)
              spawn(scope, on(sch, some_work(i)));
            // Join the work with the help of the scope
          })
        ;
    },
    make_deferred<counting_scope_resource>())
    | then([]{ std::cout << "After tasks complete\n"; });
}
```

Async Resource Lifetime Management / Async Cleanup

With the introduction of senders in P2300 there is a need to be able to represent objects that need to perform async operations during initialization and cleanup. However, these operations cannot currently be implemented using the language's constructors and destructors, as those functions cannot be asynchronous.

The async-scope facility described above is an example of such an object - it should ensure that all async operations nested within the scope are joined before the async-scope object's lifetime ends - joining all of those operations is itself an async operation and so cannot be done from a destructor.

We anticipate that there will be many more resources that require async init/cleanup and so it would be useful for the standard library to define a standard interface that objects with async initialization/cleanup can implement in a uniform way.

This would allow generic algorithms to be written to manage their lifetimes and compose these objects in a way that models the automatic nesting of object lifetimes provided by the language.

This async cleanup interface is needed in the initial release only to the extent that the async-scope facility should ideally implement this interface.

Work on a design has begun, but is not yet ready to put into a paper.

Heap-allocating sender algorithms need allocator support

There are several sender algorithms in P2300 that need to allocate storage for the state of the operation in dynamic memory.

- `ensure_started()` needs to allocate storage so that the operation executing concurrently in the background has a stable location to store the

result while still allowing the returned sender to be moved around and composed with other sender-algorithms
- `split()` needs to allocate shared-state for which all copies of the returned sender share ownership
- `start_detached()` needs to allocate storage for the operation-state of the operation launched in the background. This storage is freed upon completion of the operation.

As currently specified in P2300R7 these algorithms all allocate storage using global operator new and so it is not possible to customize the allocation strategy. If P3002 is adopted as policy then all new facilities that allocate memory should accept an allocator and use that allocator for allocation and deallocation of memory.

These algorithms need to be modified to support custom allocators. They should also be specified to use the allocator from the input sender's environment by default, if the environment specifies a value for the 'get_allocator' query.

Note that it may be possible to add support for allocator customisation later by adding an overload to these algorithms that takes an extra parameter for the allocator.

However, if we wanted to later specify these algorithms to use the allocator from the sender's attributes then that could be a potential ABI break - existing calls to these functions might start returning senders with different types/layouts

Note that there is another item listed in this document that suggests we should remove `ensure_started()`, which would obviously remove the need to add allocator support to that algorithm.

Coroutine task type

One of the key motivations for adding the coroutines language feature in C++20 was to make it easier for users to write asynchronous code using the co_await syntax. However, to date we have not yet added a coroutine-type that lets users co_await asynchronous operations.

The main reason for delaying the addition of such a coroutine type to the standard library was because we wanted to make sure that it integrated well with the async model of the rest of the standard library and so were focusing on getting the core async model in place first.

With P2300 introducing sender/receiver as the async model, we can now define an initial coroutine type that allows awaiting senders and that also presents the task as a sender.

We expect the majority of application code using sender/receiver to be written using coroutines, with sender-algorithms being used where needed to introduce concurrency.

For example: We want to be able to write the following code

```
std::execution::task<int> foo();
std::execution::task<int> bar(int x);
std::execution::task<int> baz(int x);

std::execution::task<int> example() {
 // co_await another task directly
```

```
  int x = co_await foo();

  // Calling another task-returning function and passing
tasks
  // to the when_all() sender algorithm.
  auto [a, b] = co_await std::execution::when_all(bar(x),
baz(x));

 co_return std::min(a, b);
}
```

There are some capabilities that the initial coroutine task type should include:

- The ability to `co_await` senders
- The task return-type is able to be awaited by other tasks as well as being used as a sender
- Ability to customize the coroutine-frame allocation - ideally use the same design as `std::generator`
- Transparent cancellation - a stop-request issued by a parent operation is transparently passed down to child operations awaited by the coroutine
- Scheduler-affinity - the coroutine always resumes on the associated scheduler's context after awaiting another operation
- Ability to transparently pass through a user-specified set of environment queries from the consumer of the task to child operations of the coroutine
- Avoidance of stack-overflow when awaiting synchronously-completing senders in a loop

Note that we may choose to add other coroutine types in future that provide different feature-sets for different use-cases, but this should be the default one that most developers reach for when writing async code.

There has been an implementation of a task type in stdexec that supports most of these features. It is missing support for allocator customization and has not yet incorporated a strategy for avoiding stack-overflow when awaiting synchronously-completing senders in a loop, but can otherwise form a solid basis for a standard task type.

Work is required to turn this task-type implementation into a proposal.

This paper will need to address some design questions:

- If it supports scheduler affinity, what is the default scheduler type? Should it be a type-erased scheduler? If so, one will need to be included in the proposal.
- Do we need to also include a trampoline_scheduler as a strategy for avoiding stack-overflow?
- Should we also include a nothrow-task type for tasks that cannot complete with an error?

System Execution Context (P2079)

At the moment, the only scheduler included in P2300 is from `std::execution::run_loop`, which requires manually driving its execution by calling the `run()` function, which can only be done from a single thread, once.

To allow C++ programs to make full use of the system, we should provide some kind of default, out of the box scheduler that can schedule work to take advantage of multiple CPU cores on modern processors.

While it would be relatively straightforward to implement and provide some kind of `static_thread_pool` execution context that owns its own pool of threads, we do not recommend adding such a facility to the standard library, as experience has shown that in large systems, use of such a thread-pool class tends towards each software component constructing its own thread pool of O(CPU core count) threads which then leads to 100's or even 1000's of threads being created where there are far fewer CPU cores available to execute them. This results in poor performance due to the large number of context-switches required to fairly schedule all of those threads. A Malthusian disaster!

Instead, the standard library should provide an execution context that is shared across many components within the process. Where available, the execution context should map on to the underlying operating system's built-in thread pool facilities, e.g. Windows Thread Pool or Grand Central Dispatch (GCD), so that the system can scale the thread-pool as appropriate based on available compute resources. However, it also needs to be able to work on systems that might only have a single thread of execution - in which case scheduling work onto the system context might dispatch it to the main event loop.

Shipping `std::execution` without a system execution context would severely limit the out-of-box experience for users of the facility, requiring them to pull in third-party libraries to be able to do useful work.

There are still a number of outstanding design challenges which need to be resolved in any solution:

- **Extensibility** - future releases will want to add support for additional features to the system context - time-based scheduling, async file I/O, networking, etc. We need to require that implementations of the system context have a stable ABI that allows adding features later without breaking the ABI of existing usage of the system context.
- **Replaceability** - require that the system context for a compiled binary can be replaced by an implementation not produced by the std library vendor (depends on a stable-abi), similar to how an application can replace the global `operator new/delete`.
- **Shareability** - require that the same system context can be shared across all binaries in the same process (depends on a stable ABI).
- **Lifetime** - specify when the system context is available to be used. What are its startup/shutdown constraints, and what behavior results when the system context is used outside those constraints? When do thread_local objects from thread-pool threads get destroyed with regards to [basic.start.term]?

Acknowledgements

Thanks to Inbal Levi, Gašper Ažman, and Maikel Nadolski for input and feedback on this paper.

Appendix A - A list of existing utilities in P2300

- Core concepts
  - `sender / sender_in / sender_to`
  - `receiver / receiver_of`
  - `operation_state`
  - `queryable`
  - `stoppable_token`
  - `scheduler`
- Utilities
  - `receiver_adaptor`
  - `completion_signatures`
  - `transform_completion_signatures`
- Algorithms
  - Utilities
    - `execute`
  - Sender factories
    - `just / just_error / just_stopped`
    - `read`
    - `schedule`
  - Sender adaptors
    - `on`
    - `transfer`
    - `schedule_from`
    - `then / upon_error / upon_stopped`
    - `let_value / let_error / let_stopped`
    - `bulk`
    - `split`
    - `when_all`
    - `into_variant`
    - `stopped_as_optional`
    - `stopped_as_error`
    - `ensure_started`
  - Sender consumers
    - `sync_wait`
    - `start_detached`
- Execution Contexts
  - `run_loop`
- Coroutine Utilities
  - `as_awaitable`
  - `with_awaitable_senders`